



ISA³: a 3-dimensional expansion of instance space analysis

Connor Simpson¹ · Mario Andrés Muñoz^{2,3} · Sevvandi Kandanaarachchi^{3,4} · Ricardo J. G. B. Campello⁵

Received: 20 January 2025 / Revised: 23 July 2025 / Accepted: 12 August 2025
© The Author(s) 2025

Abstract

The experimental validation of algorithms depends strongly on the characteristics of the test set used. Ideally, such a set should exhibit diverse characteristics that challenge an algorithm and are present in real-world problems. An approach to examine the diversity and representativeness of a test set is Instance Space Analysis, which uses a 2D projection to visualise the test set while identifying the strengths and weaknesses of competing algorithms. However, this has the limitation of discarding potentially useful information while crowding out the space as more features and algorithms are considered. This paper describes an extension of Instance Space Analysis into 3D, which retains a higher degree of information while maintaining the explainability of the visualisation by finding the rotations that maximise the linear trends. In addition to the expansion to 3D, a new algorithm for identifying portfolio footprints is introduced, offering a more robust and reliable method for identifying footprints in both 2D and 3D instance spaces. As a case study, we present a performance analysis of unsupervised anomaly detection methods, subject to changes in the normalisation technique. The results demonstrate the advantages by identifying regions of strength of algorithms previously thought to be underpowered.

Keywords Algorithm selection · Anomaly detection · Experimental algorithmics · Instance space analysis · Normalisation

1 Introduction

An essential step in the algorithm design process is to perform a performance assessment through an experimental study. Such an assessment facilitates, among other goals, determining the effect of components and parameters, comparing between algorithms, and identifying strengths and weaknesses (Bartz-Beielstein et al., 2020). A critical element in such studies is the selection of a test set of *diverse* and *representative* problem instances. That is, the instances exhibit, at different levels, characteristics known to be challenging to an algo-

Editor: Jan van Rijn.

Extended author information available on the last page of the article

rithm and present in real-world problems. For example, in a regression context, high values of average mutual information between predictors, a non-linear response, compounded with a low number of samples of high sparsity, often results in a challenging dataset for Gradient Boosting, but less so for Bayesian ARD (Muñoz et al., 2021).

An approach to examining the characteristics of a test set is Instance Space Analysis (ISA) (Smith-Miles and Muñoz, 2023), which represents a problem instance as a vector of features projected into a 2D plane, with the features being measures of the instance characteristics. ISA constructs the projection by maximising the linear trends in features and algorithm performance. This results in a visual representation of the complex *instance space*, which can be split into regions of good and bad algorithm performance known as *footprints*. Therefore, ISA facilitates the development of hypotheses about the algorithm's behaviour, which we can test further. Moreover, ISA helps determine whether the distribution of the instances is uniform and how such distribution relates to performance.

Nonetheless, projecting into a 2D space has the disadvantage of discarding potentially useful information. As a result, instances that otherwise could be considered different may become neighbours. Moreover, as we analyse more features and algorithms, the space becomes crowded out, and we observe fewer differentiating factors between features and algorithms. A way to address these issues is to project onto a 3D space, which retains a higher degree of information, providing additional separability while maintaining the ability to visualise. However, a challenge arises in maintaining the explainability of the visualisation. For this purpose, it is helpful to identify space rotations with the highest explanatory power, i.e., the rotations maximise the observable trends in instance features and algorithm performance.

In this paper, we describe extensions to ISA that allow us to generate such a 3D space and its associated optimal rotations while generating 3D versions of the footprints with a lesser sensitivity to outliers. We describe ISA's basic concepts and algorithms in Sect. 2.1. We then present our new algorithms in Sect. 3¹ As a case study, we present a performance analysis of unsupervised anomaly detection methods, subject to changes in the normalisation technique in Sect. 4.1. The results are described in Sect. 4. We finalise by drawing conclusions and avenues for further research in Sect. 5.

2 Background

An essential step across many Machine Learning (ML) tasks is selecting the best algorithm for a specific use case, often called the Algorithm Selection Problem (ASP) (Rice, 1976). This is a complex problem as it is common for many competing algorithms to exist, each with unique strengths and weaknesses where no single choice is best across all instances.

One popular approach to the ASP is Meta-Learning, where ML models are used to recommend or rank potential algorithms for use with a specific problem (Vilalta and Drissi, 2002; Vanschoren, 2019). These models are trained on test datasets or instances, each with meta-features describing their properties and a performance metric for each potential algorithm. From these models, meta-learning aims to recommend suitable algorithms for potential future instances. However, these methodologies are primarily black boxes that do not explain why one algorithm is better than another (Ribeiro et al., 2016). Meta-Learning is

¹ Code for the ISA³ methodology available at: <https://github.com/CSimpson4224/InstanceSpace3D>.

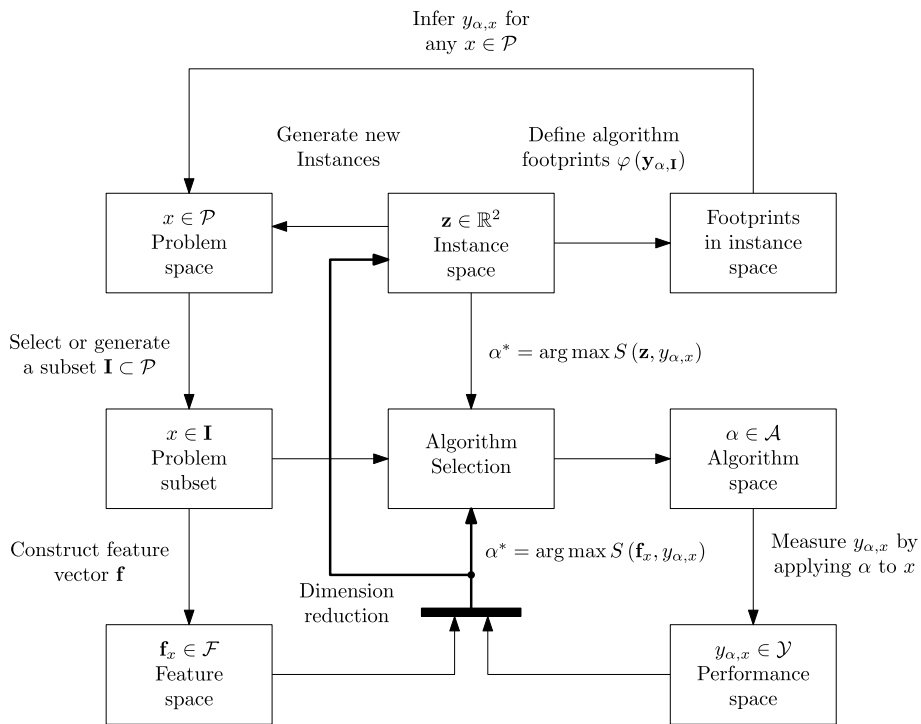
not suitable for assessing the relationships between meta-features and the performance of algorithms that determine these recommendations, which is essential to the development of future algorithms or in selecting diverse benchmark datasets.

While the goal of meta-learning is to model and predict the best algorithm for a specific task, Instance Space Analysis (ISA) approaches a different aspect of the ASP to investigate the relationships between problems or instances, their features, and the performance of algorithms for these problems (Smith-Miles and Muñoz, 2023). ISA achieves this by producing a visually interpretable model by projecting the feature space into a 2D Instance Space. ISA provides further insights not offered by traditional meta-learning methodologies, such as investigation of instance diversity and similarity, instance bias, identifying easy or difficult instances and their relationship to the features, and a deeper understanding of the relationship between the performance of each algorithm and features. In this way, ISA complements meta-learning by providing a greater understanding of a problem and may be used to develop better recommendations. As an exploratory analysis technique, ISA may also use ground-truth features, which are often avoided for meta-learning techniques. This information is not available in practice and, thus, invalid for algorithm selection. As the goal of ISA is not to produce a model for algorithm selection but instead investigate relationships between the instances and algorithms, this limitation does not apply, i.e., ground-truth features can be used to provide essential insights into the relationships between algorithms and instances, not available through other features.

Although traditional Meta-Learning techniques do not provide insights into the relationships between meta-features and algorithm performance, there do exist several methods for extracting this information, including techniques such as LIME (Ribeiro et al., 2016) and SHAP (Lundberg and Lee, 2017). These techniques help investigate the weighting or impact of each meta-feature on the prediction of a meta-learning model. These methods primarily investigate the weighting and effect of each feature on the model's output or individual prediction of a meta-learning model. ISA, in comparison, provides a more cohesive method of exploring relationships between instance features and performance through the ability to display these relationships in a single 2D space. As such, ISA enables more detailed analysis, such as identifying good and poor performance regions and the relationships between these regions to features and assessing instance diversity.

2.1 Instance space analysis

What follows is a summary of the ISA methodology (Smith-Miles and Muñoz, 2023). Figure 1 illustrates the conceptual framework behind ISA, composed of six *spaces* or sets. The *problem space*, $\mathcal{P}$, contains all the relevant instances of a problem in an application domain. From $\mathcal{P}$, we have a *subset of instances*, I , for which we have *meta-data*, i.e., data from computational experiments. The meta-data is divided into *features* and *performance*. The features describe the similarities and differences between instances in I and correlate with the performance of one or more algorithms. Features are calculated for a given instance $x \in I$, and are the elements of the vector, f_x , representing an instance in the *feature space*, $\mathcal{F}$. The performance of an algorithm α in solving an instance $x \in I$, $y_{\alpha,x}$, is an element of a vector that describes the *performance space*, $\mathcal{V}$, and requires a user-defined measure of “goodness”, such as the solution quality for a fixed computational budget. The set of algorithms available to solve the instances in I , is the *algorithm space*, $\mathcal{A}$, from which



α is a member. The meta-data corresponds to the matrices $F = [f_1 \dots f_n] \in \mathbb{R}^{m \times n}$ and $Y = [y_1 \dots y_n] \in \mathbb{R}^{a \times n}$, where m is the number of features, n is the number of problem instances, and a is the number of algorithms.

ISA uses the meta-data for two objectives. The first is to learn a selection mapping, $S(\cdot)$, which is used to identify the best algorithm α^* for a given instance x : $\alpha^* = \arg \max S(f_x, y_{\alpha, x})$. The second is to map each instance from its m -dimensional feature vector to a point z in a $2D$ *instance space*. In it, comparisons between algorithms are performed by estimating their *footprints*, $\varphi(y_{\alpha, \cdot})$, which are the regions in the instance space where we expect good performance of an algorithm. Within this conceptual framework, there are six core steps to the ISA methodology:

1. Collect the meta-data for a set of instances (I) and a portfolio of algorithms ($\mathcal{A}$), comprising feature values (F) for all instances, and a performance metric (Y) for all algorithms on all instances;
2. Construct an instance space using a feature selection process on the meta-data $\{F, Y\}$, for a user-defined measure of good performance, including its theoretical boundary;
3. Generate machine learning predictions for automated algorithm selection;
4. Generate algorithm footprints and metrics;
5. Analyse the instance space to gain insights based on current meta-data and review the sufficiency of the meta-data;
6. Generate additional meta-data if required, and repeat from Step 2; else stop.

The construction of an instance space and the algorithm footprints is performed by 5 methods, known as: (1)PRELIM, which prepares the meta-data by specifying a binary measure of “good” performance and bounding and scaling the meta-data for subsequent machine learning; (2)SIFTED, which selects a subset of $q < m$ features by considering correlations with algorithm performance and eliminating redundancies; (3)PILOT, which projects instances from high dimensional feature space to the $2D$ instance space, enforcing linear trends in features and algorithm performance distributions to ease visualisation; (4)CLOIS-TER, which approximates the instance space’s boundary using experimental or theoretical upper and lower bounds of the features; and (5) TRACE, which builds and characterises the good and best footprints for each algorithm, using their area, density and purity. This work focuses on PILOT and TRACE, which we describe below.

3 Methods

3.1 Projecting into 3D and finding optimal viewpoints

The most important algorithm in ISA is PILOT (Projecting Instances with Linearly Observable Trends), first described in (Muñoz et al., 2018). PILOT exposes the relationships between instance features and algorithm performance by constructing a $2D$ coordinate system where performance increases from one edge of the space to the opposite while enforcing linear trends on the feature distributions. To achieve this aim, PILOT finds the matrices $A_r \in \mathbb{R}^{2 \times q}$, $B_r \in \mathbb{R}^{q \times 2}$ and $C_r \in \mathbb{R}^{a \times 2}$ which minimise the approximation error of the $q + a$ linear models:

$$\|\tilde{F} - \hat{F}\|_F^2 + \|Y - \hat{Y}\|_F^2$$

where $Z = A_r \tilde{F}$, $Z \in \mathbb{R}^{n \times 2}$ is the matrix of instance coordinates in the $2D$ space, $\tilde{F} \in \mathbb{R}^{q \times n}$ is the matrix of reduced number of features obtained after SIFTED, $\hat{F} = B_r Z$, and $\hat{Y} = C_r Z$. We assume that $q < n$ and $\tilde{F}$ is full row rank, i.e. $\text{rank}(\tilde{F}) = q$. Thus, PILOT numerically solves the problem:

$$\begin{aligned} \min \quad & \|\tilde{F} - B_r Z\|_F^2 + \|Y - C_r Z\|_F^2 \\ \text{s.t. } Z = & A_r \tilde{F} \\ (\mathcal{D}) \quad & A_r \in \mathbb{R}^{2 \times q} \\ & B_r \in \mathbb{R}^{q \times 2} \\ & C_r \in \mathbb{R}^{a \times 2} \end{aligned} \quad (1)$$

using BFGS (Broyden, 1970), by representing $\{A_r, B_r, C_r\}$ as a column vector. Since BFGS always finds a global optimum for $\mathcal{D}$, the final solution is the one with the highest topological preservation, i.e., the correlation between the distances in the feature, $\|f_i - f_j\|$, and instance, $\|z_i - z_j\|$, spaces is the highest from N_{try} repeats.

Generating a $3D$ projection using PILOT is relatively straightforward by redefining the problem in Eq. (1) by setting $Z \in \mathbb{R}^{n \times 3}$, $A_r \in \mathbb{R}^{3 \times q}$, $B_r \in \mathbb{R}^{q \times 3}$ and $C_r \in \mathbb{R}^{a \times 3}$. However, the challenge arises in finding an optimal viewpoint, i.e., a location for the camera that

maximises the visible trends for a single or a group of features or algorithms. Assuming that the distance to the origin of the space is constant, the problem of finding the viewpoint corresponds to finding optimal azimuth and rotation angles. To do so, we define the following optimisation problem:

$$\begin{aligned} \min & \|Y - VZ\|_F^2 + \lambda \langle v_1, v_2 \rangle \\ \text{s.t. } & Z = A_r \tilde{F} \end{aligned} \quad (2)$$

where $V = [v_1 \ v_2] \in \mathbb{R}^{3 \times 2}$ is a matrix that takes the 3D space coordinates Z and *flattens* them into a plane where the linear trends in Y are being maximised. To guarantee that V defines a rotation, it must be orthonormal. We achieve orthogonality by adding the dot product of the column vectors of V as a penalty function in Eq. (1), multiplied by a scaling factor λ with a default value of 0.2 used to reduce its impact on the optimisation problem. As the projection is solved numerically, if the cost for orthogonality is significantly higher than that of the projection angle, the solver may find suboptimal angles prioritising only orthogonality. Through experimental calibration, we found that a scaling factor of $\lambda = 0.2$ had minimal impact on the solver while ensuring orthogonality was met. The sensitivity of this parameter is low with values below $\lambda = 0.5$ providing similar results, and should not require adjustment. To guarantee normality, both column vectors are independently rescaled by their magnitude. To determine the azimuth and rotation angles, we find the orthogonal vector defined by V through the cross product of its vectors. The problem is solved numerically using BFGS as it is done to find Z .

Depending on the needs, we may use this process to find the viewpoints to analyse a single algorithm or a smaller group of algorithms. As such, it eliminates the trade-off produced by trying to find a 2D projection that linearises all features and algorithms at once.

3.2 TRACE³: from area to volume footprints with less sparseness

The second most important algorithm in ISA is TRACE (Triangulation with Removal of Areas with Contradicting Evidence), first described in (Muñoz and Smith-Miles, 2017). TRACE constructs algorithm *footprints*, which are the generalised area of the instance space where good (or best) performance is expected based on data. Footprints provide depth to performance assessment by considering the instance characteristics, their diversity and possible bias, and the strength and unambiguousness of the empirical evidence. Their area, density and purity characterise footprints.

TRACE follows three core steps: (a) estimating the area and density of the convex hull containing all instances in I , which are used as baseline metrics to normalise each algorithm's footprint as a percentage of the instance space; (b) building and characterising the footprints for each algorithm; and (c) comparing the best footprints to reduce or eliminate overlapping sections with weak evidence. To construct an algorithm's footprint, TRACE uses DBSCAN (Ester et al., 1996) to identify dense clusters of good (and best) performing instances, which are then enclosed by α -shapes, a generalisation of the concept of the convex hull from computational geometry (Edelsbrunner et al., 1983).

Whenever two algorithms claim to be the best in an area of the instance space, their footprints overlap. Overlapping sections with lower purity are removed from the footprint

to remove ambiguity. If the purity is the same for both footprints, the overlapping section is kept due to insufficient evidence of either algorithm's dominance.

A weakness of TRACE is its reliance on DBSCAN to detect dense regions in the space, becoming prone to identifying sparse regions as dense in noisy instance spaces. This is because DBSCAN as an unsupervised clustering technique, does not use the performance labels directly, with TRACE instead using them after obtaining the clusters. Therefore, we have developed a new footprint algorithm based on K-Nearest Neighbours classification (KNN), which is able to function for both 2D and 3D instance spaces. KNN was selected as the classification method because it closely aligns with the underlying intuition behind portfolio footprints: regions with similar feature values are expected to exhibit similar performance. By using KNN, we can determine whether an instance lies within a region of good performance by assessing whether the majority of its neighbourhood also demonstrates good performance. Although this new footprint algorithm has been constructed from the ground up, it follows a similar structure to the original TRACE. TRACE³ follows these core steps, which will be further elaborated on below: (a) estimating the area and density of the convex hull containing all instances in I , which are used as baseline metrics to normalise each algorithm's footprint as a percentage of the instance space; (b) construct a KNN classifier for each algorithm (c) build α -shape footprints from the KNN classifier predictions (d) trim outlier and sparse regions of the footprints.

As the goal of the footprint algorithm is to highlight distinct regions of good performance in the space, we aim to construct a conservative classifier that only identifies instances as good if there is sufficient evidence. For this, we carefully adjusted the two input parameters to KNN, i.e., the number of nearest neighbours (k) and the prior weighting on the classification, to conservative values. These parameters' selection and impact on the produced footprints are further discussed in Sect. 4.2. For k , a reasonably large value should be used to ensure outlier or low-density good instances near bad performance instances are not classified as "good" by the KNN algorithm and, as a default, $k = 50$ has been selected.

On the other hand, we used a 40%-60% prior for identifying instances as good and bad, respectively. This ensures that noisy regions are only marked as good if they contain a definite majority of good instances, i.e., acting as an initial purity constraint. This prior can be tightened if a more conservative classification is required.

Input: A matrix $\mathbf{Z} \in \mathbb{R}^{n \times d}$ ($d \in \{2, 3\}$) of coordinates in the instance space, a vector $\mathbf{y} \in \mathbb{B}^{n \times 1}$ of binary performance measures, and options **Opts**

Output: A footprint Φ and summary statistics

```

1: function BUILDFOOTPRINT( $\mathbf{Z}, \mathbf{y}, \text{Opts}$ )
2:    $\mathcal{M}_{KNN} \leftarrow \text{fitcknn}(\mathbf{Z}, \mathbf{y}, \text{'Prior'}, \text{Opts.Prior}, \text{'NumNeighbors'}, \text{Opts.NN})$   $\triangleright$ 
   Train KNN model
3:    $\hat{\mathbf{y}} \leftarrow \text{predict}(\mathcal{M}_{KNN}, \mathbf{Z})$   $\triangleright$  Predict labels
4:    $\mathbf{Z}_u \leftarrow \{\mathbf{z}_i \in \mathbf{Z} \mid \hat{\mathbf{y}}_i = 1 \wedge \mathbf{y}_i = 1\}$   $\triangleright$  Retain correctly predicted good instances
5:    $\Phi.\text{polygon} \leftarrow \text{BuildAlphaShap}(\mathbf{Z}_u)$   $\triangleright$  Build initial  $\alpha$ -shape
6:    $\Phi.\text{area} \leftarrow \text{Area}(\Phi.\text{polygon})$ 
7:    $\Phi.\text{count} \leftarrow \text{CountElements}(\Phi.\text{polygon}, \mathbf{Z})$ 
8:    $\Phi.\text{good} \leftarrow \text{CountElements}(\Phi.\text{polygon}, \{\mathbf{z}_i \mid \mathbf{y}_i = 1\})$ 
9:    $\Phi.\text{density} \leftarrow \Phi.\text{count} / \Phi.\text{area}$ 
10:   $\Phi.\text{purity} \leftarrow \Phi.\text{good} / \Phi.\text{count}$ 
11:   $\mathbf{A}_S \leftarrow \text{AlphaSpectrum}(\Phi.\text{polygon})$   $\triangleright$  Get alpha radii producing distinct shapes
12:   $i \leftarrow 1$ 
13:  while  $\Phi.\text{purity} < \text{Opts.Purity} \wedge i \leq |\mathbf{A}_S|$  do  $\triangleright$  Iterate trimming until purity
   threshold is met, or the footprint area/volume is reduced to 0.
14:     $\Phi.\alpha \leftarrow \mathbf{A}_S[i]$ 
15:     $\Phi.\text{RegionThreshold} \leftarrow \Phi.\text{area} / 20$   $\triangleright$  Suppress small disconnected regions
16:     $\Phi.\text{good} \leftarrow \text{CountElements}(\Phi.\text{polygon}, \{\mathbf{z}_i \mid \mathbf{y}_i = 1\})$ 
17:     $\Phi.\text{count} \leftarrow \text{CountElements}(\Phi.\text{polygon}, \mathbf{Z})$ 
18:     $\Phi.\text{density} \leftarrow \Phi.\text{count} / \Phi.\text{area}$ 
19:     $\Phi.\text{purity} \leftarrow \Phi.\text{good} / \Phi.\text{count}$ 
20:     $i \leftarrow i + 1$ 
21:  end while
22: end function

```

Algorithm 1 TRACE³: footprint construction algorithm

The footprints are constructed utilising α -shapes, which are generated by setting a radius around each point, or vertex, of the shape where points intersected by this radius will be connected by an edge to create a bounding area or volume. The default radius for α -shapes is the minimum such that all points are included in the shape. Only instances for which the original and predicted labels agree are utilised for constructing the footprints.

As footprints represent an area of good performance by a specific algorithm, we expect most instances within a footprint to exhibit good performance and, thus, a purity greater than 0.5. The primary cause of footprints not achieving this purity is outlier points extending the footprint over large areas of bad instances that should be excluded from the footprint. This is solved by iteratively reducing the radius used to construct the α -shape to exclude outlier points, starting from the minimum required to encircle all instances, until a purity threshold is met (with a default of 0.6) or no α -shape is produced. On each step of this iteration, a “region threshold” parameter is set to reduce the volume of the footprint by 5% of the previous iteration, which aims to discard small sections containing outlier points. Pseudocode for the TRACE³ algorithm can be seen in Algorithm 1.

4 A case study on unsupervised anomaly detection

4.1 Meta-data for anomaly detection

The advantage of a 3D projection becomes apparent when many algorithms need to be analysed. For this purpose, we revisit the experiment described by Kandanaarachchi et al. (2020) on unsupervised anomaly detection, which is the task of identifying out-of-distribution *observations* without relying on training-set labels. Anomaly detection is a core task due to its relevance to real-world scenarios such as cyber-attacks and stock market crashes.

The anomaly detection instances within this study consist of tabular datasets, with *predictors*² represented as columns, $\mathbf{x}$, with a_x and b_x being statistics drawn from each column. When the predictors have different scales, e.g., the height and weight of a person in centimetres and kilograms, these need to be normalised to prevent the large-scale predictors from dominating the detection method. Normalisation is then defined as the transformation $\frac{\mathbf{x} - a_x}{b_x}$, which directly alters the distance between observations. Hence, the accuracy of an anomaly detection method may depend on the normalisation method. Kandanaarachchi et al. (2020) explored this effect using four normalisation techniques and 14 anomaly detection algorithms, resulting in 56 algorithm variants. The four normalisation methods were:

1. Min-Max with $a_x = \min(\mathbf{x})$, $b_x = \max(\mathbf{x}) - \min(\mathbf{x})$.
2. Mean-SD with $a_x = \text{mean}(\mathbf{x})$, $b_x = \text{sd}(\mathbf{x})$.
3. Median-IQR with $a_x = \text{median}(\mathbf{x})$, $b_x = \text{IQR}(\mathbf{x})$.
4. Median-MAD with $a_x = \text{median}(\mathbf{x})$, $b_x = \text{MAD}(\mathbf{x})$ where $\text{MAD}(\mathbf{x}) = \text{median}(|\mathbf{x} - \text{median}(\mathbf{x})|)$.

While the 14 anomaly detection methods were taken from (Campos et al., 2016):

1. KNN: K nearest neighbours (Ramaswamy et al., 2000)
2. KNNW: KNN weight (Angiulli and Pizzuti, 2002)
3. ODIN: Outlier Detection using In-degree Number (Hautamaki et al., 2004)
4. LOF: Local Outlier Factor (Breunig et al., 2000)
5. Simplified LOF: (Schubert et al., 2014b)
6. COF: Connectivity-based Outlier Factor (Tang et al., 2002)
7. INFLO: Influenced Outlierness (Jin et al., 2006)
8. LOOP: Local Outlier Probabilities (Kriegel et al., 2009)
9. LDOF: Local Density-based Outlier Factor (Zhang et al., 2009)
10. LDF: Local Density Factor (Latecki et al., 2007)
11. KDEOS: Kernel Density Estimation Outlier Score (Schubert et al., 2014a)
12. FAST ABOD: Fast Angle Based Outlier Detection (ABOD), faster version of ABOD (Kriegel et al., 2008)
13. iForest: Isolation Forest (Liu et al., 2008)
14. An ensemble using LOF, LDOF and LOOP

²We differentiate between predictors, which are the variables on individual anomaly detection instances, and features, which describe the characteristics of a complete instance.

Except for iForest, these methods rely on a k -nearest neighbourhood, resulting in a common parameter k . Noting that no single value of k is applicable for all algorithms and instances, for a given instance, the same k was used for all methods as follows:

$$k(\text{dataset}) = \min(\lfloor N/20 \rfloor, 100), \quad (3)$$

where N is the number of observations. Here, the maximum of $k = 100$ limits the amount of computation for large instances. On the other hand, the low bound of 5% of observations given by $\lfloor N/20 \rfloor$ is justified by Kandanaarachchi et al. (2020) as giving a size sufficiently large that the method would not miss outliers in small clusters, as it would be with a $k \rightarrow 1$. While potentially suboptimal for some combinations, this value offers a good trade-off.

Eight algorithm variants were chosen for an instance space analysis by Kandanaarachchi et al. (2020) due to the lack of “physical room” in the 2D projection to represent all 56 variants with sufficient nuance. The subset of instances, I , consisted of 12335 datasets derived from classification problems transformed into anomaly detection ones by down-sampling observations from every class. Moreover, an extensive set of 346 features, categorised into groups in Table 1, was calculated. Some features require normalisation with either Min-Max or Median-IQR. Seven features were used for the instance space. Finally, the performance metric was the area under the ROC Curve (AUC ROC), with a threshold of good performance of AUC ROC > 0.8 . Instance sets for which none or most of the algorithms perform well are less informative than those for which only some algorithms perform well. For this reason, in this study, we have focused only on 4247 datasets for which less than 50% (1 to 28) of the algorithms perform well.

The results by Kandanaarachchi et al. (2020) on 12,338 datasets showed that Fast-ABOD-min-max had the largest footprint area, followed by KNN-median-IQR. The algorithm variants Ensemble-median-IQR, LOF-min-max and iForest-median-IQR had similar algorithm footprints. The algorithms KDEOS-median-IQR, KDEOS-min-max and LDF-min-max had the smallest footprint areas. Using the ISA result for algorithm selection showed that KDEOS had a niche in the instance space where none of the other algorithms performed well. Moreover, this analysis concluded that LOF-min-max and Ensemble-median-IQR had small footprints. Note that this conclusion, based on the regular 2D ISA approach, had originally contradicted the findings of Campos et al. (2016), in which LOF was a robust algorithm. This is one of the aspects that we will revisit and reconsider with ISA³ in the following section.

Table 1 A breakdown of the 346 features used by Kandanaarachchi et al. (2020). Normalisation-based features require that the dataset is normalised with either Min-Max or Median-IQR before being calculated

Feature category	No. features	Normalization methods	Total features
Standard meta-learning	25	NA	25
Density-based	77	2	154
Residual based	35	2	70
Graph-based	41	2	82
Normalization based	15	NA	15
Total			346

4.2 Sensitivity of TRACE³ to its parameter settings

In Sect. 3.2, we identified several parameters in TRACE³ that must be tuned to create tighter or looser footprints for a specific instance space. These are the number of nearest neighbours, k , the prior weighting for the classification, and the minimum purity threshold for a footprint (PI).

The value of k should be selected based on the number of instances in the space. While this parameter is less sensitive as the number of instances increases, a higher value tends to avoid overweighting single or insignificant groups of good instances within areas of poor performance. Such outliers may lead to misleading results. Figure 2 shows three footprints produced for the same data using $k = \{10, 50, 200\}$, with otherwise identical settings. For $k = 10$, several small regions with few good instances were classified as good, causing small offshoots and overfitting of the footprint. Moreover, larger areas of good performance covered small regions with a high density of bad instances. For $k = 200$, TRACE³ missed a region of good performance, as highlighted by the examples with 10 and 50 nearest neighbours, respectively. Finally, for $k = 50$, TRACE³ identified small but significant regions while avoiding outlier cases. It is noteworthy that despite the large change in parameter setting, these three footprints are mostly identical with minor differences.

The prior adjusts the weight the KNN classifier gives each class, modifying the number of neighbours required to consider a majority and is set using the format: *prior* = [*BadWeighting*, *GoodWeighting*]. Using a prior with a low weighting for identifying instances as “good” will result in footprints that have a higher density and purity. The prior is Fig. 3 illustrates footprints for priors of [0.5, 0.5], [0.6, 0.4] and [0.8, 0.2] respectively, where the latter priors represent a higher weighting towards identifying instances as “bad” rather than “good”. We observe a larger footprint for a [0.5, 0.5] prior, with several areas of lower density of good instances included. For a conservative prior of [0.8, 0.2], we see a smaller footprint excluding several regions of the instance space but with increased purity and density. While this is a low-sensitivity parameter, it effectively provides a level of control over the desired the purity of footprints.

Finally, the minimum purity threshold, PI, determines the minimum acceptable purity for a footprint, removing outlier instances and low-density regions from the footprint. This parameter is ineffective if no outlier points are present or large portions of the footprint

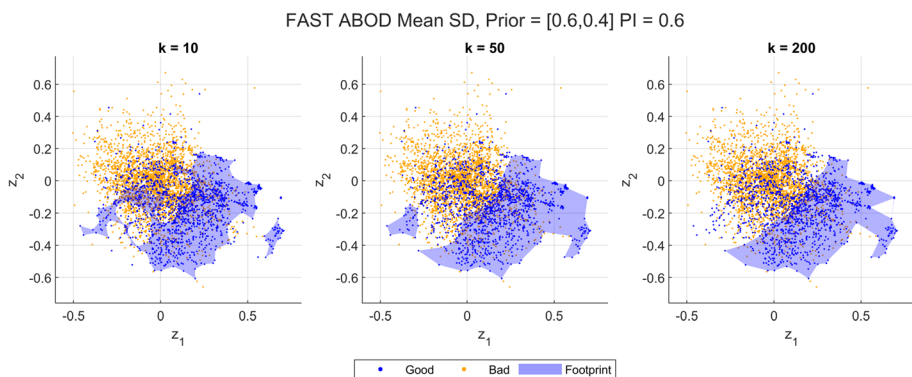


Fig. 2 Three examples of footprints produced in 2D for the TRACE³ algorithm with 10, 50 and 200 nearest neighbours, respectively

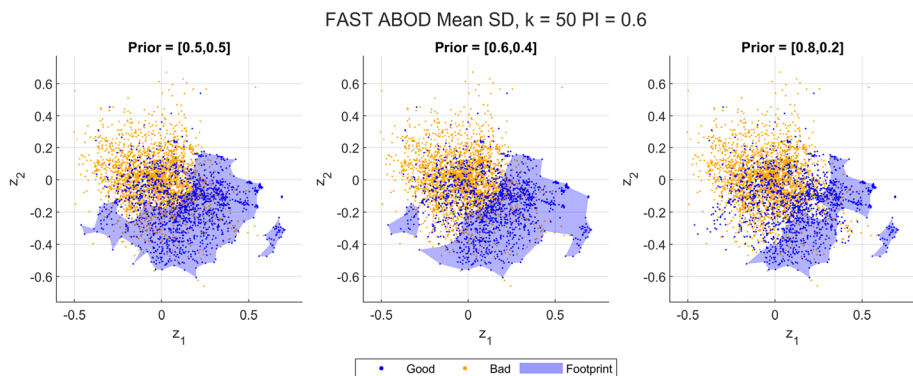


Fig. 3 Three examples of footprints produced in 2D for the TRACE³ algorithm with priors of $[0.5, 0.5]$, $[0.6, 0.4]$, and $[0.8, 0.2]$ respectively

are low-density. Therefore, it should be adjusted if the prior is unsuccessful in producing the desired purity and should be at least equal to the weight for “good” in the prior, as it will have little to no impact if below this weight. Figure 4 shows an example with $PI = \{0.4, 0.6, 0.8\}$ for a prior of $[0.6, 0.4]$, and $PI = 0.8$ with a prior of $[0.8, 0.2]$. For $PI = 0.4$, several outlier points significantly increased the footprint size by including low-density and empty regions of the instance space and increasing PI without decreasing the prior weight for “good” results in an overfitted footprint with holes within. However, matching PI to the increased prior produces a more reasonable footprint, only highlighting high-purity regions without holes.

4.3 Visualising the space

Feature selection was performed using the SIFTED algorithm, where ten features were selected, each of which are described in detail in Table 2. Although only seven features were selected for the instance space within Kandanaarachchi et al. (2020), that study only assessed eight of the fifty-six algorithm variants. As all fifty-six algorithm variants are being assessed within our study, the number of selected features was increased to ten in order to effectively capture the differences between the additional algorithms. Both the 2D and 3D instance spaces were produced by PILOT with $N_{\text{try}} = 30$ initialisations, where the best for each was kept. Further details on the ISA parameters are available in the project repository (Simpson, 2025).

Equation (4) presents the projection equation obtained with PILOT³, which takes ten features selected by SIFTED and projects them into the 3D space. Observing the weights for each feature, we can identify that $OPO_GDeg_Out_Mean_1$ is the most influential on the z_1 axis, $Num_Attributes$ is on the z_2 axis, and $OPO_Out_ResOut_1_3$ on the z_3 . Figure 5 shows the feature distributions for the 2D and 3D instance spaces. While the majority of features are seen to be accurately captured and displayed linearly by the 2D instance space, both $Num_Attributes$ and $Ratio_Obs_2_Attr$ do not, as they appear to be increasing in the direction of the axis parallel to the 2D plane. On the other hand, all features are displayed with greater separation and linearity in the 3D instance space.

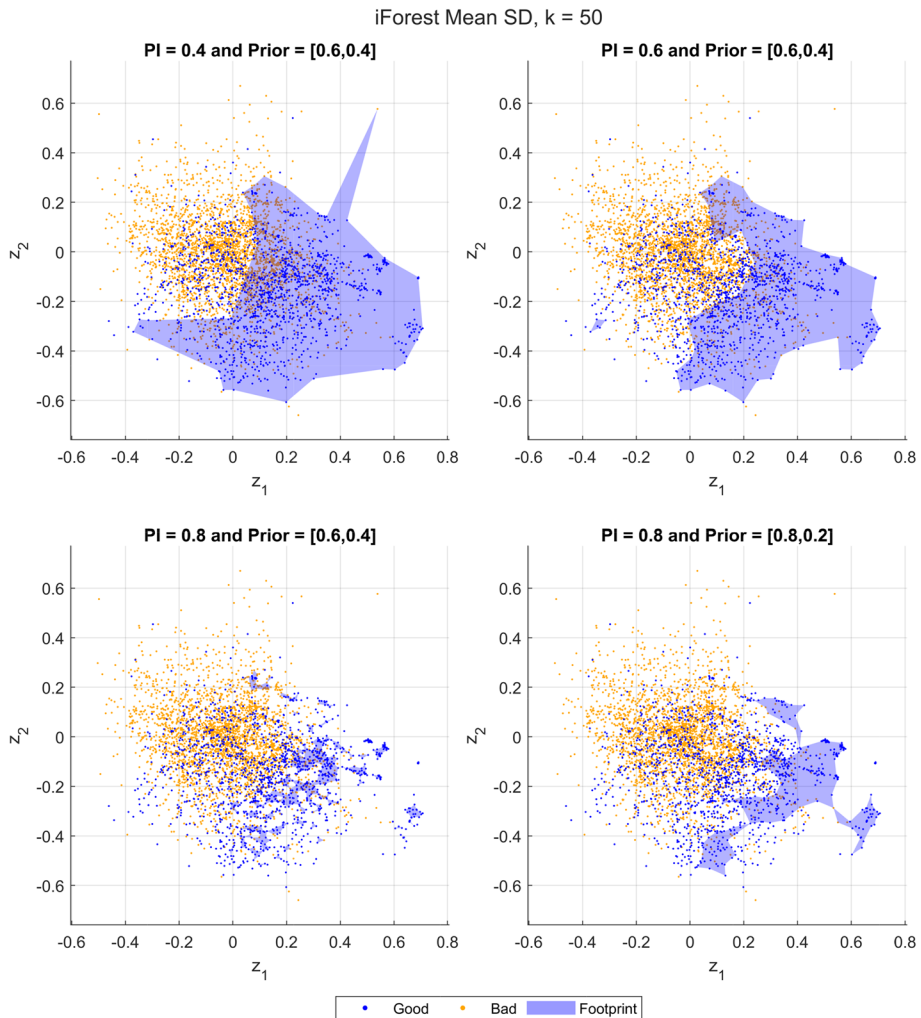


Fig. 4 Four examples of footprints produced in 2D for the TRACE³ algorithm. Three with purity thresholds of 0.4, 0.6, 0.8 each with a prior of [0.6, 0.4], and one example with a purity threshold of 0.8 and a prior of [0.8, 0.2]

Four anomaly detection algorithms have been selected that best display the differences between the previous 2D methods and the updated 3D ones presented in this paper, with the remainder of the algorithms available in the supplementary material in the project repository (Simpson, 2025). These algorithms are Ensemble, Fast ABOD, KDEOS and LOF, using the four normalisations. Figure 6, shows a 2D instance space with footprints for each of the four selected algorithms and four normalisation methods, produced by TRACE in blue, and TRACE³ in red. Comparing the two methods of generating footprints, we observe that TRACE³ appears to identify dense regions of good instances better and more closely follow the shape of the instance space. In comparison, TRACE frequently failed to identify dense regions of good instance for noisy data; instead, it identified outliers and sparse regions

Table 2 Features selected by SIFTED for the 3D instance space and their descriptions

Feature	Description
Num_Attributes	Number of attributes in the dataset
Ratio_Obs_2_Attr	Ratio of rows to columns (observations to attributes) in the dataset
OPO_Out_KNOut_1_1_x	Ratio of KNN distances between outliers to non-outliers using Min-Max normalising
OPO_Out_DenOut_1_3	Ratio of densities between outliers to non-outliers using Median-IQR normalising
OPO_Den_Out_IQR_3	The IQR of outliers density using Median-IQR normalisation
OPO_Res_ResOut_05P_3	After fitting linear models to each attribute, the ratio between the 0.05 percentile of residuals of proxy-outliers to the 0.05 percentile of residuals of non-proxy outliers is computed using Median-IQR normalising
OPO_Out_ResOut_1_3	The proportion of proxy-outliers to outliers in terms of linear model residuals using Median-IQR normalisation
OPO_Res_Out_Mean_3	After fitting linear models to each attribute, the ratio between the mean of residuals of outliers to the mean of residuals of non-outliers is computed using Median-IQR normalising
OPO_GDeg_Out_Mean_1	After converting the data to a graph, the ratio of the mean degree of outliers to mean degree of non-outliers using min-max normalising is computed
Mean_PROD_MM	Mean of the ratio of nearest neighbour distances between outliers and non-outliers

as good. A primary example of this can be seen for Fast ABOD, where TRACE³ correctly identifies dense regions of good instances. At the same time, TRACE fails to capture large portions of the dense regions, instead primarily covering outliers. Looking at LOF with Mean SD, we can see one issue with TRACE³, which may over-fit footprints by avoiding poor-performing instances among good-performing instances. However, here we see TRACE is also flawed, highlighting sparse regions of outliers. For TRACE³, stricter hyper-parameters may be used to correct these issues. However, this requires balance and may negatively impact the footprints of other algorithms. Both TRACE and TRACE³ appear to miss some potential regions of good performance, such as areas towards the bottom of the instance space for both Ensemble Mean-SD and Min-Max normalisation, and LOF with Min-Max normalisation; however, upon close inspection, these regions do also feature a large quantity of “bad” instances. This is a case where the purity of these regions is close to or below the threshold and may be captured with more relaxed constraints if the trade-off to purity is considered acceptable for the application.

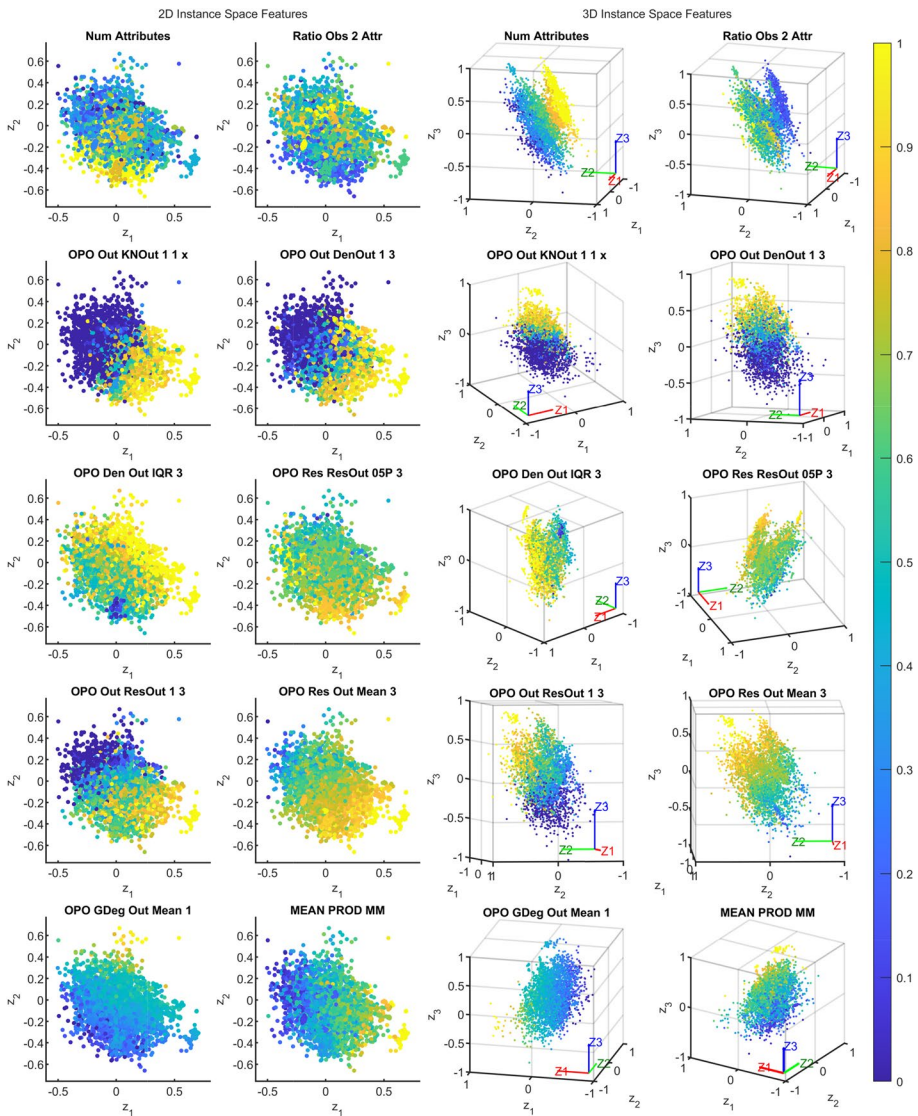


Fig. 5 Feature distributions for the traditional 2D instance space (left) and 3D instance space rotated for optimal viewing (right), where the value of each feature is represented by the colour, ranging from blue (low) to yellow (high). A compass is added to each 3D figure placed near the corner with minimum z_1 , z_2 , and z_3 values

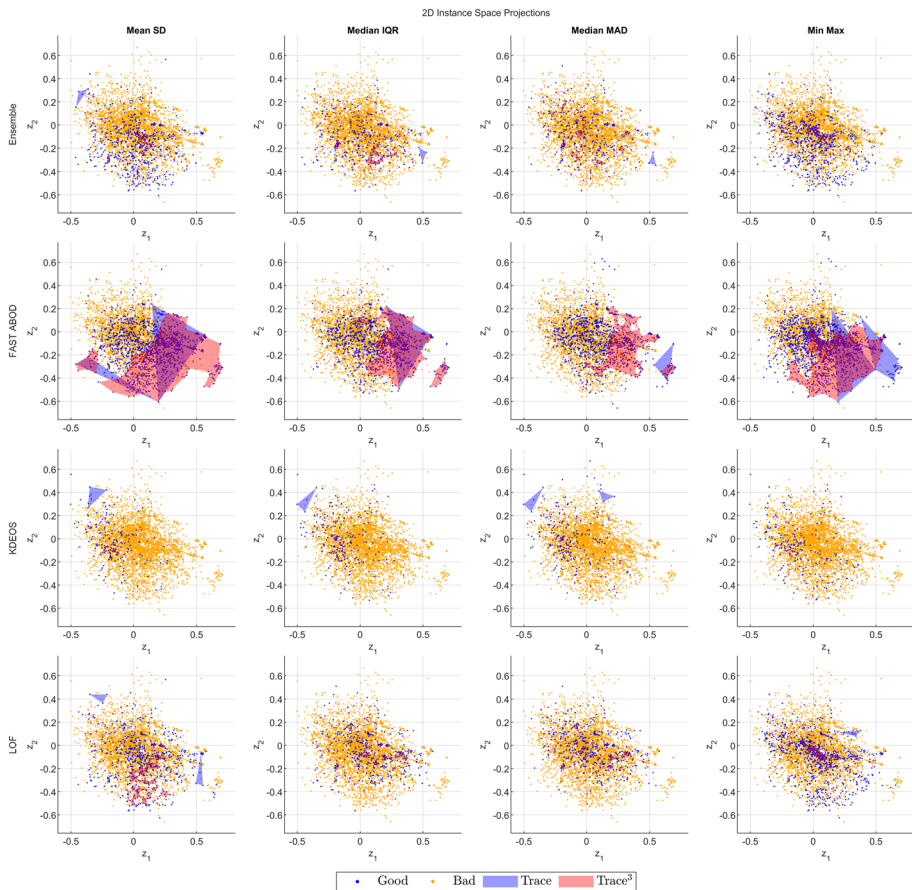


Fig. 6 Performance of four algorithms with each normalisation method in a 2D instance space with footprints from TRACE in blue and TRACE³ in red

$$Z = \begin{bmatrix} 0.0132 & -0.1108 & 0.0927 \\ 0.0351 & 0.0521 & 0.0020 \\ 0.0099 & 0.0103 & 0.0658 \\ 0.0151 & 0.0173 & 0.0712 \\ 0.0678 & 0.0430 & 0.0084 \\ -0.0336 & 0.0157 & 0.0806 \\ -0.0488 & 0.0428 & 0.0408 \\ -0.0143 & 0.0803 & 0.0950 \\ 0.1440 & -0.0983 & -0.0230 \\ 0.0591 & -0.0007 & 0.0530 \end{bmatrix}^T \begin{bmatrix} \text{Num_Attributes} \\ \text{Ratio_Obs_2_Attr} \\ \text{OPO_Out_KNOut_1_1_x} \\ \text{OPO_Out_DenOut_1_3} \\ \text{OPO_Den_Out_IQR_3} \\ \text{OPO_Res_ResOut_05P_3} \\ \text{OPO_Out_ResOut_1_3} \\ \text{OPO_Res_Out_Mean_3} \\ \text{OPO_Gdeg_Out_Mean_1} \\ \text{MEAN_PROD_MM} \end{bmatrix} \quad (4)$$

Figure 7 shows a 3D instance space for the four selected algorithms for each normalisation method. The angle of viewing the instance space is determined through the methods described above in Sect. 3.1, for which we have grouped the algorithms by normalisation method, producing four rotations.³

³The full 3D instance space for each footprint may be viewed and manually rotated through matlab using the code available in the projects repository (Simpson, 2025)

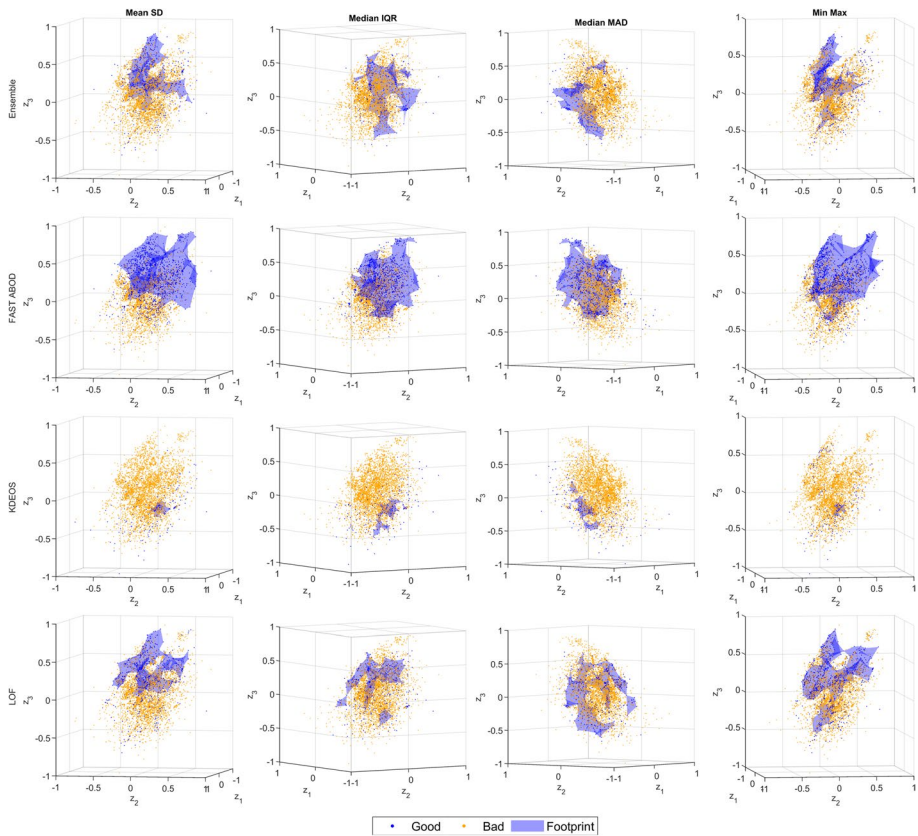


Fig. 7 A 3D instance space with footprints from TRACE³ in blue

Compared to the 2D instance space, we see a much more distinct separation of good and bad instances within our space, significantly reducing the noisiness of the instance space. Consequently, better-defined footprints are produced for all algorithms. This can be particularly useful for the three algorithms Ensemble, KDEOS and LOF, for which few poorly defined footprints were produced in a 2D instance space by both footprint algorithms. Now, using 3D instance spaces, we can identify clear and defined regions of good performance.

Figure 7 shows that the rotation method has effectively displayed the algorithms for assessment. A clear example of this is Fast ABOD with Mean SD normalisation; if viewed showing each axis equally, the footprint would appear to cover the entire space, while with the rotation, it has been appropriately displayed to show both regions of good instances covered by the footprint, and bad instances not covered.

Moreover, observing the angle of projections for the performance of algorithms in Fig. 7 in conjunction with Eq. (4), we can note that z_3 has been prioritised for all four normalisation methods, indicating features with large magnitudes in this dimension are the most important to each algorithms performance regardless of normalisation method. For Mean-SD and Min-Max z_2 is the next most important dimension with z_1 contributing little, while for Median-IQR and Median-MAD z_1 and z_2 contribute roughly equally to the projections.

We can infer from this that features with a large magnitude in z_1 such as `OP O_GDeg_Out_Mean_1` are less important to the performance of algorithms when either Mean-SD or Min-Max normalisation is used; while algorithms with similar magnitudes and directions in z_1 and z_2 such as `OP O_Den_Out_IQR_3` and `Ratio_Obs_2_Attr` will be less important for Median IQR and Median MAD normalisations as we are observing the space along this vector.

Due to a combination of using a different set of instances, updates to the code for feature selection in the instance space methodology (SIFTED), and the selection of ten features in comparison to the previous seven, there are slight differences in the features used between this study and the previous study in Kandanaarachchi et al. (2020). The Spearman correlation between the seven features selected in the previous study and the ten features selected in this study can be seen in Table 3. The feature `OPO_Res_Out_Mean_3` is the only common feature in both studies. However, four of the features selected in the original study: `OPO_Out_LocDenOut_2_3`, `PO_Res_ResOut_95P_1`, `OPO_GDeg_Out_Mean_3`, `Mean_Prod_IQR`, are similar to the following features selected within this study: `OPO_Out_Den_Out_1_3`, `OPO_Res_ResOut_05P_3`, `OPO_GDeg_Out_Mean_1` and `Mean_Prod_MM`, respectively.

4.4 Characterising the footprints

To compare the footprints within the instance space, we characterised them using three metrics: the normalised area/volume, the percentage of the total space occupied by each footprint; the normalised density, the number of instances per unit area/volume relative to the overall density of the space; and finally the purity, the percentage of good instances within each footprint. These metrics are shown in Table 4 for each of the footprints for the three examples of a 2D instance space with TRACE, 2D instance space with TRACE³, and the 3D space with TRACE³. The first result visible from this table is that while TRACE produces

Table 3 Spearman Correlation between features selected in Kandanaarachchi et al. (2020) (columns), and in this study (rows)

	OPO GComp PO Mean 1	OPO Out LocDenOut 2_3	OPO GDeg Out Mean 3	OPO Res ResOut 95P_1	OPO Res ResOut Mean 3	OPO Res Out Mean 3	MEAN PROD IQR
Num Attributes	0.460	0.120	−0.042	−0.054	0.120	−0.054	0.066
Ratio Obs 2 Attr	−0.190	−0.036	0.120	0.310	0.110	0.046	−0.031
OPO Out KOut 11.x	0.065	0.580	−0.220	0.180	0.170	0.500	0.440
OPO Out Den Out 1_3	0.170	0.820	−0.220	0.097	0.130	0.420	0.390
OPO Den Out IQR 3	−0.170	0.048	−0.016	0.100	0.095	0.110	−0.020
OPO Res ResOut 05P_3	−0.097	0.061	−0.110	0.640	0.810	0.120	−0.240
OPO Out ResOut 1_3	−0.066	0.470	−0.310	0.150	0.120	0.760	0.340
OPO Res Out Mean 3	−0.12	0.46	−0.32	0.17	0.15	1.00	0.38
OPO GDeg Out Mean 1	0.100	−0.230	0.740	−0.033	−0.098	−0.330	−0.120
MEAN PROD MM	0.230	0.510	−0.095	−0.280	−0.290	0.360	0.690

Table 4 Footprint normalised density, normalised area/volume and purity as percentages of the space for each algorithm using the three methods: a 2D instance space with TRACE, a 2D instance space with TRACE³, and a 3D instance space with TRACE³

Algorithm	2D TRACE			2D TRACE ³			3D TRACE ³		
	Density	Area	Purity	Density	Area	Purity	Density	Volume	Purity
COF_Mean_SD	0.47	0.4	50	7.9	1.5	60	4.6	4.3	60
COF_Median_IQR	0	0	0	8.6	0.9	67	5.9	2.6	60
COF_Median_MAD	0	0	0	8.8	0.8	62	5.2	3.1	60
COF_Min_Max	0	0	0	7.3	2.2	63	3.8	6.1	60
Ensemble_Mean_SD	0.22	0.6	80	6.9	0.8	62	3.6	3.9	60
Ensemble_Median_IQR	0.18	0.4	100	3.2	1.3	63	2.6	3.6	62
Ensemble_Median_MAD	0.29	0.3	75	3.7	1.3	62	2.6	2.8	60
Ensemble_Min_Max	0	0	0	7.9	0.7	62	3.9	4.2	60
FAST_ABOD_Mean_SD	0.84	24	63	0.91	34	69	1.3	24	72
FAST_ABOD_Median_IQR	0.73	13	66	1.3	19	60	1.9	15	61
FAST_ABOD_Median_MAD	0.22	2.4	96	1.4	13	60	2.3	12	61
FAST_ABOD_Min_Max	0.77	22	59	1.5	26	60	1.9	23	65
iForest_Mean_SD	0.36	30	70	0.93	30	60	1.2	26	61
iForest_Median_IQR	0.32	21	78	0.95	29	60	1.4	21	60
iForest_Median_MAD	0.38	18	79	1.1	26	62	1.1	28	61
iForest_Min_Max	0.37	18	73	0.99	27	60	1.3	23	60
INFLO_Mean_SD	0	0	0	1.6	2.3	60	4.1	1.3	62
INFLO_Median_IQR	0	0	0	5.8	0	67	3.1	0.2	62
INFLO_Median_MAD	0	0	0	6.9	0.1	60	3.5	0.4	62
INFLO_Min_Max	0	0	0	9.1	0.3	61	6	1.3	60
KDEOS_Mean_SD	0.18	1.5	64	5.8	0.4	65	14	0.1	64
KDEOS_Median_IQR	0.11	1.1	80	7.3	0.2	69	8	0.4	62
KDEOS_Median_MAD	0.14	1.7	70	8.4	0.2	60	7.8	0.5	62
KDEOS_Min_Max	0	0	0	7.6	0.2	72	13	0.1	67
KNN_Mean_SD	0.43	21	83	1.3	30	64	1.5	29	62
KNN_Median_IQR	0.74	29	67	1.6	19	64	2.2	14	71
KNN_Median_MAD	0.41	17	86	1.6	21	62	1.8	19	64
KNN_Min_Max	0.89	19	64	1.4	29	68	2.3	17	75

Notable results discussed within the text are displayed in bold. Algorithms where TRACE³ outperforms TRACE in all categories for the 2D instance space have been highlighted

no footprint for several algorithms, TRACE³ produces some footprint in almost all cases. We can, however, note that in several cases, the footprints for TRACE³ cover few points and should frequently be disregarded. This can also be seen in the area; for example, for KDEOS, TRACE³ produces footprints with areas lower than 0.5% normalised area. This, however, is still a preferable behaviour to TRACE, which, for KDEOS, produced footprints of low density, only containing outlier points.

We see significantly higher normalised density when comparing the two footprint algorithms for the traditional 2D instance space. Notably, all footprints produced by TRACE have a normalised density lower than 1, indicating these footprints have a lower density than the total for the space. In contrast, the normalised density for TRACE³ is greater than 1 for most algorithms, indicating it has successfully found regions with a high density of good instances. We frequently observe a higher area for TRACE³, although many exceptions exist where the original algorithm produces a higher area, such as for the KDEOS algorithm. However, it should be noted that where TRACE has produced a higher area, the density is significantly lower compared to TRACE³. These cases are generally TRACE highlighting a large area only containing few outliers as seen in Fig. 6 for the KDEOS algorithm.

We often see TRACE producing higher purity footprints than TRACE³. Although higher purity results may be desired, for the instance space produced, the dense regions of good instance can often be seen in Fig. 6 to contain numerous bad instances, which TRACE has avoided, resulting in higher purity at the cost of missing these regions. We see comparable purity in both methods for algorithms where TRACE has produced reasonable footprints,

Table 5 (Continued) Footprint normalised density, normalised area/volume and purity as percentages of the space for each algorithm using the three methods: a 2D instance space with TRACE, a 2D instance space with TRACE³, and a 3D instance space with TRACE³

Algorithm	2D TRACE			2D TRACE ³			3D TRACE ³		
	Density	Area	Purity	Density	Area	Purity	Density	Volume	Purity
KNNW_Mean_SD	0.46	29	77	1.2	38	61	2.2	16	75
KNNW_Median_IQR	0.16	3.4	91	1.7	19	63	1.8	21	60
KNNW_Median_MAD	0.33	11	69	1.7	18	60	1.7	20	63
KNNW_Min_Max	0.29	16	87	1.4	28	72	2.5	15	77
LDF_Mean_SD	0.26	6.7	78	2.3	8	60	2.7	8.2	61
LDF_Median_IQR	0.25	3.6	65	3.7	3.5	61	3.4	5.9	61
LDF_Median_MAD	0.16	2.5	100	3.4	3.9	60	3.8	4.7	61
LDF_Min_Max	0.28	4.2	94	2.6	6.1	60	3	7.5	61
LDOF_Mean_SD	0.29	5.1	57	3.9	1.7	63	3.2	3.9	60
LDOF_Median_IQR	0	0	0	4.2	1.2	66	3.3	2.4	60
LDOF_Median_MAD	0	0	0	5.6	0.7	61	2	4.3	60
LDOF_Min_Max	0.49	0.2	60	3.2	1.8	62	2.5	4.6	61
LOF_Mean_SD	0.14	1.5	67	2.6	3.3	61	3.6	4.8	60
LOF_Median_IQR	0	0	0	6.2	0.9	62	3.9	2.4	61
LOF_Median_MAD	0	0	0	5.4	0.9	60	3.2	3.9	60
LOF_Min_Max	0.5	0.5	80	8.4	1.1	61	3.5	6.1	60
LOOP_Mean_SD	0.34	7.5	56	6.5	0.8	62	3.2	4.3	62
LOOP_Median_IQR	0	0	0	3.8	1.5	60	2.7	3.5	61
LOOP_Median_MAD	0	0	0	4.5	1	61	3	2.7	62
LOOP_Min_Max	0	0	0	8	0.6	64	3.9	4.4	60
ODIN_Mean_SD	0.35	3.4	63	2.7	2.4	61	3.1	3.5	61
ODIN_Median_IQR	0.39	0.7	91	3.7	1.3	60	4	1.8	60
ODIN_Median_MAD	0	0	0	2.7	1.9	62	6.1	1.2	61
ODIN_Min_Max	0.21	1.8	75	8	0.6	60	3.6	4	61
SIMLOF_Mean_SD	0.28	4.7	57	7.4	0.6	67	3.1	4.8	60
SIMLOF_Median_IQR	0	0	0	5.2	1	61	2.8	3.7	63
SIMLOF_Median_MAD	0.29	0.3	75	5.3	0.9	63	2.6	4.3	60
SIMLOF_Min_Max	0.23	4.2	60	6.9	1.2	60	3	6.3	60

Algorithms where TRACE³ outperforms TRACE in all categories for the 2D instance space have been highlighted

such as for Fast ABOD with both Mean SD and Median IQR normalisation. However, the density and area for these cases are greater for TRACE³.

As we normalise the area/volume and density, these values should be comparable between the 2D and 3D instance space. Comparing the results of TRACE³ between a 2D and 3D instance space, we see little difference between them. The density of footprints between 2D and 3D are generally comparable; however, unlike for the 2D footprints, all densities for the 3D footprints are greater than 1. This further indicates better separation between good and bad performance regions in a 3D instance space. The normalised volumes of the 3D footprints are generally larger than the areas of the 2D instance space, while in most cases, the purity is comparable between the two. However, this is likely due to how the new algorithms achieve the desired purity thresholds. Given this, it is possible the purity threshold could be further increased for 3D spaces in comparison to the 2D instance space to produce denser footprints with higher purity than achievable in 2D.

In order to provide a quantitative assessment of the footprints produced by each TRACE method, and TRACE³ in 2D vs 3D, we will treat the footprints as classifiers and assess their ability to identify good performance while excluding bad performance. To evaluate this performance, a 10-fold cross-validation procedure is employed, where the F-score is calculated to assess how well the footprint captures regions of good performance. The mean F-score for the four algorithms previously shown in Sect. 4.3 is shown in Table 6, comparing the original TRACE algorithm on the 2D instance space with TRACE³, applied to both 2D and 3D instance spaces. The results for all algorithm variants are available in the supplementary

Table 6 The F-Score for 16 variants calculated from the 10-fold cross validation

Algorithm	F-Score		
	2D TRACE	2D TRACE ³	3D TRACE ³
Ensemble_Mean_SD	0.019	0.108	0.324
Ensemble_Median_IQR	0.000	0.147	0.231
Ensemble_Median_MAD	0.000	0.108	0.145
Ensemble_Min_Max	0.005	0.114	0.347
FAST_ABOD_Mean_SD	0.430	0.619	0.588
FAST_ABOD_Median_IQR	0.199	0.512	0.490
FAST_ABOD_Median_MAD	0.214	0.413	0.434
FAST_ABOD_Min_Max	0.294	0.584	0.586
KDEOS_Mean_SD	0.016	0.102	0.078
KDEOS_Median_IQR	0.000	0.043	0.148
KDEOS_Median_MAD	0.007	0.049	0.159
KDEOS_Min_Max	0.000	0.020	0.033
LOF_Mean_SD	0.002	0.136	0.331
LOF_Median_IQR	0.004	0.145	0.255
LOF_Median_MAD	0.005	0.155	0.301
LOF_Min_Max	0.017	0.236	0.421

material. It is important to note when considering these results that the objective of the footprint algorithms differs from that of traditional classifiers. Rather than aiming for perfect classification accuracy, the goal is to conservatively identify regions where performance exceeds a specified threshold, in this case, regions with more than 60% good performance.

From Table 6, comparing TRACE and TRACE³ for a 2D instance space, we can see a significantly higher F-Score for TRACE³ for all algorithms. This indicates the algorithm is better in both its ability to identify good regions and to exclude bad regions. For Ensemble, KDEOS and LOF, we observe low F-Scores, as there were few regions of good performance to be captured by footprint algorithms, regardless of the algorithm used.

Comparing TRACE³ between a 2D and 3D space in Table 6, we can see the F-Score is higher for most algorithms, with the exceptions of FAST_ABOD_Mean_SD and FAST_ABOD_Median_IQR. For these two algorithm variants, this is likely a factor due to FAST_ABOD variants already being well explained by a 2D space, unlike other methods, resulting in less benefit being provided by the increase to 3D. This could also be seen in FAST_ABOD_Median_MAD and FAST_ABOD_Min_Max, which presented similar results in both the 2D and 3D spaces. Despite the FAST_ABOD variants not seeing an increase in performance, they still present the best performance in the 3D instance space. The increase in performance between the 2D and 3D instance space is most evident for Ensemble and LOF, where the 3D case significantly outperforms the 2D case by a wide margin. These results demonstrate the advantages of a 3D space, allowing for greater separability of good and bad regions to produce more accurate insights.

4.5 Analysing new evidence on algorithm performance

A significant advantage of ISA³ is that we can explore more normalisation-anomaly detection combinations due to the 3D space having more *room*. For example, Kandanaarachchi et al. (2020) selected eight algorithm variants out of 56, using a clustering analysis

Table 7 Best anomaly detection algorithms for each normalisation method

Normalisation method	Algorithms giving best performance
Min-Max	COF, Ensemble, INFLO, LDOF, LOF, LOOP, ODIN, SIMLOF
Median-IQR	KNNW
Median-MAD	iForest, KDEOS
Mean-SD	FAST-ABOD, INFLO, KNN, LDF

to discover groups of algorithms with correlated performance. These were (a) Ensemble-Median-IQR, (b) LOF-Min-Max, (c) KNN-Median-IQR, (d) FAST-ABOD-Min-Max, (e) iForest-Median-IQR, (f) KDEOS-Median-IQR, (g) KDEOS-Min-Max and (h) LDF-Min-Max; In this subset, no algorithm variants contain COF, INFLO, KNNW, LDOF, LOOP, ODIN or SIMLOF. Using ISA³, we do not need to select a handful of algorithms; instead, in this study, we used the complete set of instances for which less than half of the algorithms were good performing. Previously absent variants had 3D footprints of comparable size to the eight originally included, while others remained weak in both versions. For example, COF variants have similar volumes to LOF; INFLO had no area in 2D but a small volume in 3D; LDOF, LOOP and SIMLOF, which are LOF variants, give similar results to each other in 3D, with SIMLOF surpassing LOF for two normalisation methods; ODIN gave similar weak results both in 2D and 3D; finally, KNNW had a footprint that was similar to KNN in both 2D and 3D. These algorithms were previously left out of the instance space due to the limitations in the 2D methodology; however, now with ISA³, we have the ability to include and fairly compare all algorithms, leading to new insights as were seen for SIMLOF.

From Fig. 7, we see that Median-IQR and Median-MAD sometimes give rise to patch-like footprints spread over a wider region (see Ensemble and LOF). In contrast, Min-Max and Mean-SD give rise to contiguous footprints, an effect also observable for other algorithms not displayed in Fig. 7 like LOOP and ODIN (see supplementary material (Simpson, 2025)). These differences can be explained by how different normalisation can impact anomaly detection. Specifically, normalising methods Min-Max and Mean-SD bring the outliers closer to the non-outlying data, whereas Median-IQR and Median-MAD do not, as they are robust to outliers. As a result, the anomaly detection methods behave noticeably differently on datasets subject to different normalisation strategies. Thus, two datasets close by in the feature space may not have similar performances for different normalisation techniques, making it important to assess these methods with respect to the normalisation used. In contrast, datasets that are further apart in the feature space can perform well for an algorithm when normalised using Median-based methods, as the anomalies are not brought in towards the rest of the data. This can result in patch-like footprints.

Kandanaarachchi et al. (2020) observed similarities in summary performance values between Min-Max and Mean-SD and between Median-IQR and Median-MAD. We see further evidence of this in Fig. 7. Moreover, different normalisation methods achieve the largest footprint for each algorithm. Table 7 presents the preferred normalisation method for each algorithm. For example, Min-Max is the most popular method, followed by Mean-SD. Median-IQR and Median-MAD are less popular. Most LOF variants prefer Min-Max.

In terms of algorithm performance, we observe in Fig. 6 that Ensemble and LOF have no or insignificant footprints in 2D; however, Fig. 7 shows that both of them had significant footprints for all normalisation methods. The result for LOF is noteworthy since the 2D analysis had originally contradicted the results by Campos et al. (2016), in which LOF

performed well. Figure 7 demonstrates that the LOF footprint has a significant, contiguous region of the space that performs well, demonstrating that it is a competitive algorithm.

While there are differences in algorithm footprints between 2D and 3D instance spaces, there are also similarities. FAST-ABOD is a strong algorithm with a large footprint in 2D and 3D instance spaces. Similarly, KDEOS is an algorithm with a small footprint in 2D and 3D instance spaces. Overall, adding another dimension enhances the 2D instance space, and we can see its effect in algorithm footprints.

5 Conclusions

Instance Space Analysis (ISA) is a methodology that facilitates the analysis of experimental data to evaluate the diversity and representativeness of an instance test set while exposing the strengths and weaknesses of competing algorithms. However, ISA is limited by its use of a 2D projection that, although it facilitates the visualisation and interpretation of results, discards potentially useful information while crowding out the space as more features and algorithms are considered. We described two modifications to the most critical algorithms in ISA, PILOT and TRACE, to generate and operate in a 3D projection. As such, higher information is retained while maintaining the explainability of the visualisation by finding the rotations that maximise the linear trends. We tested the results on an unsupervised anomaly detection case study. As a result, we identified regions of strength of algorithms previously thought to be underpowered.

Further work on ISA³ includes implementing non-linear projections to generate the instance space. We can consider Eq. 1 to train a type of auto-encoder with linear inputs and outputs. These can be extended to other types of functions, which may be able to better separate the feature data while retaining a level of interpretability. Examining the differences between our proposed framework and the results provided in MATILDA (Smith-Miles et al., 2020), may result in identifying further regions of strength in those scenarios. Finally, exploring scenarios available in ASLib, would help us further evaluate the generalizability of ISA³.

Author contributions All authors contributed to the study's conception and design. Connor Simpson and Mario Andrés Muñoz developed the methodology, and Connor Simpson created the software. Sevvandi Kandanaarachchi performed data collection and preparation. Connor Simpson, Mario Andrés Muñoz, and Sevvandi Kandanaarachchi carried out the analysis and wrote the first draft. Ricardo J. G. B. Campello provided critical review and feedback on the manuscript and analysis of results. All authors read and approved the final manuscript.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions

Data availability Supplementary material, data and scripts used to produce this study are available at <https://github.com/CSimpson4224/ISA3DAnomalyDetection>. Code for the ISA³ methodology is available at <https://github.com/CSimpson4224/InstanceSpace3D>.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License,

which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Angiulli, F., & Pizzuti, C. (2002). Fast outlier detection in high dimensional spaces. In T. Elomaa, H. Mannila, & H. Toivonen (Eds.), *Principles of data mining and knowledge discovery* (pp. 15–27). Springer.
- Bartz-Beielstein, T., Doerr, C., van den Berg, D., Bossek, J., Chandrasekaran, S., Eftimov, T., & Weise, T. (2020). Benchmarking in optimization: Best practice and open issues. <https://arxiv.org/abs/2007.03488>
- Breunig, M. M., Kriegel, H. P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (pp. 93–104). Association for Computing Machinery.
- Broyden, C. G. (1970). The convergence of a class of double-rank minimization algorithms 1. General considerations. *IMA Journal of Applied Mathematics*, 6(1), 76–90. <https://doi.org/10.1093/imamat/6.1.76>
- Campos, G. O., Zimek, A., Sander, J., Campello, R. J., Mícenková, B., Schubert, E., & Houle, M. E. (2016). On the evaluation of unsupervised outlier detection: Measures, datasets, and an empirical study. *Data Mining and Knowledge Discovery*, 30(4), 891–927. <https://doi.org/10.1007/s10618-015-0444-8>
- Edelsbrunner, H., Kirkpatrick, D., & Seidel, R. (1983). On the shape of a set of points in the plane. *IEEE Transactions on Information Theory*, 29(4), 551–559. <https://doi.org/10.1109/TIT.1983.1056714>
- Ester, M., Kriegel, H., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining* (pp. 226–231). AAAI Press.
- Hautamaki, V., Karkainen, I., & Franti, P. (2004). Outlier detection using k-nearest neighbour graph. In *Proceedings of the 17th international conference on pattern recognition* (3, pp. 430–433). IEEE.
- Jin, W., Tung, A. K., Han, J., & Wang, W. (2006). Ranking outliers using symmetric neighborhood relationship. In *Pacific-Asia conference on knowledge discovery and data mining* (pp. 577–593). Springer.
- Kandanaarachchi, S., Muñoz, M. A., Hyndman, R. J., & Smith-Miles, K. (2020). On normalization and algorithm selection for unsupervised outlier detection. *Data Mining and Knowledge Discovery*, 34(2), 309–354. <https://doi.org/10.1007/s10618-019-00661-z>
- Kriegel, H. P., Kröger, P., Schubert, E., & Zimek, A. (2009). LoOP: Local outlier probabilities. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management* (pp. 1649–1652). Association for Computing Machinery.
- Kriegel, H. P., Schubert, M., & Zimek, A. (2008). Angle-based outlier detection in high-dimensional data. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 444–452). Association for Computing Machinery.
- Latecki, L. J., Lazarevic, A., & Pokrajac, D. (2007). Outlier detection with kernel density functions. In *International workshop on machine learning and data mining in pattern recognition* (pp. 61–75). Springer.
- Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. In *2008 Eighth IEEE international conference on data mining* (pp. 413–422).
- Lundberg, S., & Lee, S. I. (2017). A unified approach to interpreting model predictions. [arXiv:1705.07874](https://arxiv.org/abs/1705.07874)
- Muñoz, M., & Smith-Miles, K. (2017). Performance analysis of continuous black-box optimization algorithms via footprints in instance space. *Evolutionary Computation*, 25(4), 529–554. <https://doi.org/10.1162/evco.a.00194>
- Muñoz, M., Villanova, L., Baatar, D., & Smith-Miles, K. (2018). Instance spaces for machine learning classification. *Machine Learning*, 107(1), 109–147. <https://doi.org/10.1007/s10994-017-5629-5>
- Muñoz, M. A., Yan, T., Leal, M. R., Smith-Miles, K., Lorena, A. C., Pappa, G. L., & Rodrigues, R. M. (2021). An instance space analysis of regression problems. *ACM Transactions on Knowledge Discovery from Data*. <https://doi.org/10.1145/3436893>
- Ramaswamy, S., Rastogi, R., & Shim, K. (2000). Efficient algorithms for mining outliers from large data sets. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (pp. 427–438). Association for Computing Machinery.

- Ribeiro, M.T., Singh, S. & Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. [arXiv:1602.04938](https://arxiv.org/abs/1602.04938)
- Rice, J. R. (1976). The algorithm selection problem. *Advances in Computers*, 15, 65–118.
- Schubert, E., Zimek, A. & Kriegel, HP. (2014a). Generalized outlier detection with flexible kernel density estimates. In *Proceedings of the 2014 SIAM international conference on data mining* (pp. 542–550).
- Schubert, E., Zimek, A. K., & H, P. (2014b). Local outlier detection reconsidered: A generalized view on locality with applications to spatial, video, and network outlier detection. *Data Mining and Knowledge Discovery*, 28(1), 190–237. <https://doi.org/10.1007/s10618-012-0300-z>
- Simpson, C. (2025). Scripts and meta-data for ISA3D + anomaly detection. Available at <https://github.com/CSimpson4224/ISA3DAnomalyDetection>.
- Smith-Miles, K., & Muñoz, M. A. (2023). Instance space analysis for algorithm testing: Methodology and software tools. *ACM Computing Surveys*, 55(1), 21–31. <https://doi.org/10.1145/3572895>
- Smith-Miles, K., Muñoz, M., Neelofar, N. (2020). Melbourne algorithm test instance library with data analytics (MATILDA). Available at <https://matilda.unimelb.edu.au>.
- Tang, J., Chen, Z., Fu, A. W. C. & Cheung, D. W. (2002). Enhancing effectiveness of outlier detections for low density patterns. In *Pacific-Asia conference on knowledge discovery and data mining* (pp. 535–548). Springer.
- Vanschoren, J. (2019). Meta-learning meta-learning. In F. Hutter, L. Kotthoff, & J. Vanschoren (Eds.), *Automated Machine Learning: Methods, Systems, Challenges* (pp. 35–61). Springer.
- Vilalta, R., & Drissi, Y. (2002). A perspective view and survey of meta-learning. *Artificial Intelligence Review*, 18(2), 77–95. <https://doi.org/10.1023/A:1019956318069>
- Zhang, K., Hutter, M. & Jin, H. (2009). A new local distance-based outlier detection approach for scattered real-world data. In *Pacific-Asia conference on knowledge discovery and data mining* (pp. 813–822). Springer.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Connor Simpson¹ · Mario Andrés Muñoz^{2,3} · Sevvandi Kandanaarachchi^{3,4} · Ricardo J. G. B. Campello⁵

✉ Connor Simpson
connor.simpson@newcastle.edu.au

Mario Andrés Muñoz
munoz.m@unimelb.edu.au

Sevvandi Kandanaarachchi
sevvandi.kandanaarachchi@data61.csiro.au

Ricardo J. G. B. Campello
campello@imada.sdu.dk

¹ School of Information and Physical Sciences, The University of Newcastle, University Drive, Callaghan, NSW 2308, Australia

² School of Computing and Information Systems, The University of Melbourne, Parkville, Melbourne, VIC 3010, Australia

³ ARC Centre in Optimisation Technologies, Integrated Methodologies, and Applications, OPTIMA, 700 Swanston Street, Carlton, VIC 3053, Australia

⁴ Data61, CSIRO, Research Way, Clayton, VIC 3168, Australia

⁵ Department of Mathematics and Computer Science, University of Southern Denmark, Campusvej 55, 5230 Odense, Denmark